AI-Assisted Test Planning, Edge Cases and Test Data

LESSON SPECIFICATIONS & ROADMAP

Deconstruct complex user stories, discover hidden edge cases with adversarial AI prompting, and engineer safe synthetic data factories.

• Track: Playwright Automation with JavaScript + AI • Level: Advanced
• Estimated Effort: 2.5–3 Hours • Prerequisites: Days 1–17 concepts (Locators, web-first assertions, POM, API testing, and f
• Runtime: Pure Modern JavaScript (.js) • Runner: @playwright/test

Day 18 Learning Objectives

'Translate ambiguous PRDs and user stories into a structured, risk-weighted P0/P1/P2 scenario matrix.
'Overcome confirmation bias using adversarial AI prompts to discover concurrency, timezone, and boundary flaws.
'Design safe Test Data Factories generating dynamic, isolated synthetic fixtures using the RFC-reserved @example.com domain.
'Eliminate production PII and security compliance risks from test repositories.
'Implement living requirements traceability in Playwright using tags and annotations.
'Execute targeted requirement verification suites via Playwright CLI grep filtering.

Software requirements are frequently delivered as brief Jira tickets or vague user stories: "As a traveler, I want to book a hotel room with discounts so that I can save money."

If an automation engineer writes tests directly from this single sentence, critical edge cases are guaranteed to be overlooked:
- What happens if the check-out date is before the check-in date?
- What happens if room inventory sells out between selection and payment?
- Can a promotional coupon discount exceed the room subtotal?
- What currency rounding rules apply to international taxes?

Using AI as a critical thinking partner, you can deconstruct ambiguous requirements into an exhaustive, risk-weighted test scenario matrix:
- P0 (Blocker / Critical): Core revenue workflows (Search, Select, Pay, Confirm). Total loss of revenue if broken.
- P1 (High): Validation rules, boundary dates, error notifications. User frustration if broken.
- P2 (Medium / Low): Obscure coupon combinations, extreme character inputs, localization.

Priority Tiered Describe Block (Organizing P0, P1, and P2 tests into descriptive Playwright test blocks.)

```
test.describe('Hotel Reservation Suite', () => {
  test('P0: completes standard room booking for valid future dates', async ({ page }) => {
    // Core revenue flow
  });
  test('P1: prevents reservation when check-out precedes check-in', async ({ page }) => {
    // Validation rule
  });
});
```

Human developers and QA engineers suffer from confirmation bias: we instinctively test the paths we expect users to follow.

AI models, when prompted with adversarial framing ("Act as an ethical hacker or rigorous QA architect trying to break this state machine"), excel at uncovering unconventional paths:
1. Boundary & Off-By-One Conditions: Leap years, Daylight Saving Time (DST) switches, timezone crossings, midnight reservations.
2. State Machine Violations: Double-clicking "Submit Payment", refreshing the page during credit card authorization, navigating backwards in browser history.
3. Concurrency Collisions: Two users simultaneously reserving the very last available hotel room or seat.
4. Data Extreme Boundaries: 0 items, 9999 items, zero-dollar transactions, strings containing SQL injection probes or multi-byte Unicode emojis.

Testing Rapid Multi-Click Prevention (Simulating rapid double clicks to verify single request dispatch.)

```
const submitButton = page.getByRole('button', { name: 'Confirm Reservation' });
await submitButton.click({ clickCount: 2 });
// Verify single request dispatch via request event counter
```

Many enterprise teams make the dangerous mistake of copying real production database dumps into test environments. This introduces massive legal and operational vulnerabilities:
- GDPR / CCPA Violations: Exposing real customer names, phone numbers, and addresses.
- Accidental Customer Contact: Tests accidentally firing SMS notifications or real billing charges to genuine customers.
- Stale State: Customer data changes or is deleted, causing brittle tests to fail unpredictably.

Safe Synthetic Data Generation Strategies:
Modern Playwright frameworks generate fresh, isolated synthetic data for every test execution using dynamic timestamps, random suffixes, and the official RFC-reserved "@example.com" domain. This guarantees zero state collision during parallel test runs.

Booking Data Factory Pattern (Generating isolated booking fixtures with dynamic future dates and unique emails.)

```
export function generateBookingData(overrides = {}) {
  const timestamp = Date.now();
  const randomSuffix = Math.floor(Math.random() * 10000);
  return {
    guestName: `Guest ${randomSuffix}`,
    email: `guest_${timestamp}_${randomSuffix}@example.com`,
    ...overrides
  };
}
```

Requirements Traceability is the architectural practice of linking every automated test directly back to the business requirement, user story, or acceptance criteria (AC) it verifies.

Without traceability:
- Teams cannot answer: "If we ship this release, what percentage of our PRD is verified by automation?"
- Refactoring requirements causes orphan tests to linger or critical features to lose coverage without anyone noticing.

Implementing Traceability in Playwright:
Playwright provides native test annotations and tag capabilities to embed requirement IDs directly into test executions and HTML reports:
test('AC-HOTEL-104: applies 20% loyalty discount', { tag: ['@req-104', '@loyalty'] }, async ({ page }) => { ... })
You can then run targeted tests using the CLI: npx playwright test --grep "@req-104".

Tagging Test with Requirement ID (Using the tag property to link test execution directly to Jira acceptance criteria.)

```
test('REQ-HOTEL-02: enforces minimum 1-night stay', {
  tag: ['@req-02', '@boundary'],
}, async ({ page }) => {
  await page.goto('/reserve');
  // test logic
});
```

requirements.

Assignment Starter Code (Day 18)

```
// ============================================================================
// ROOTACCESS PLAYWRIGHT AUTOMATION — DAY 18 PRACTICAL LAB
// AI-Assisted Test Planning, Edge Cases & Synthetic Data
// ============================================================================
// Scenario: Hotel & Flight Booking Platform ("RootVoyage")
// Requirements:
// - REQ-BK-01: Valid booking requires future check-in and checkout > checkin.
// - REQ-BK-02: Maximum occupancy validation (Family Suite: max 4 guests).
// - REQ-BK-03: Promotional coupon code application (Code: "SAVE20" gives 20% off).
// - REQ-BK-04: Negative date boundary validation (checkout <= checkin rejected).
// ============================================================================

import { test, expect } from '@playwright/test';

export function generateSyntheticBookingData(overrides = {}) {
  const timestamp = Date.now();
  const randomSuffix = Math.floor(1000 + Math.random() * 9000);

  const checkIn = new Date();
  checkIn.setDate(checkIn.getDate() + 10);
  const checkOut = new Date(checkIn);
```

Q1. How do you design an automated test suite from an ambiguous product specification or user story?

[Difficulty: Intermediate] Evaluation Focus: Requirements analysis, risk categorization, and test scoping

Model Answer: I begin by deconstructing the user story into explicit acceptance criteria using a 3-tier risk matrix (P0 revenue-critical, P1 negative/boundary validation, P2 edge cases). I collaborate with product owners or use structured AI prompts to uncover missing constraints—such as off-by-one dates, concurrency conflicts, and boundary extremes. Each identified scenario is then given a unique requirement ID (e.g. @req-101), authored into an independent Playwright test using synthetic data factories, and tagged for living traceability in reports.

Follow-Up Question: "What questions would you ask a Product Manager when a feature spec says "users can apply coupon codes"?"

Q2. Why is using real production database data in automated testing considered an anti-pattern, and what is the alternative?

[Difficulty: Beginner] Evaluation Focus: Data privacy, security compliance, and test data management

Model Answer: Using production data creates severe legal and regulatory liabilities under GDPR/CCPA due to customer PII exposure. Furthermore, test environments risk accidentally sending real SMS/emails or charging real cards. Operationally, production data is stateful and fragile—if a customer account changes, tests fail unpredictably. The industry standard alternative is the Test Data Factory pattern, which generates isolated, synthetic records with unique timestamps and standard reserved domains (@example.com) per test run.

Follow-Up Question: "How do you handle tests that require existing catalog data (e.g. 5,000 products) without copying production?"

Q3. What is Requirements Traceability, and how can Playwright natively support it?

[Difficulty: Intermediate] Evaluation Focus: Traceability, test reporting, and enterprise governance

Model Answer: Requirements Traceability connects automated test cases directly to the business requirements they validate, ensuring teams know their exact automated test coverage. Playwright natively supports traceability through test tagging (e.g. tag: ["@req-bk-01"]) and the annotations API (test.info().annotations.push({ type: "requirement", description: "REQ-01" })). These tags and annotations appear in the official HTML report and allow targeted execution via npx playwright test --grep "@req-bk-01".

Follow-Up Question: "How does tagging help in a continuous delivery environment where only smoke tests run on PRs?"

Q4. How do you test for race conditions like duplicate order submission when a user rapidly double-clicks a submit button?

[Difficulty: Scenario-Based] Evaluation Focus: Concurrency testing, API event interception, and UI state debouncing

Model Answer: I simulate the rapid interaction using Playwright's locator click options, such as await button.click({ clickCount: 2 }) or dispatching rapid successive clicks. Simultaneously, I monitor outgoing network requests using page.on("request") to verify that only a single POST request is dispatched to the backend, and assert that the button enters a disabled or loading state immediately upon the initial click.

Follow-Up Question: "How should the frontend architecture guard against rapid multi-clicks?"

Q5. How does an engineer prevent AI tools from creating unrealistic or overly simplistic test data?

[Difficulty: Scenario-Based] Evaluation Focus: AI prompt design, data boundary testing, and schema constraints

Model Answer: By providing rigorous constraints in the prompt: specifying domain-valid regex patterns, realistic boundary conditions (e.g. international phone formats, Unicode characters in names, leap year dates), and explicit relational rules (e.g. child checkout date must exceed checkin date). Rather than accepting static JSON fixtures, I prompt AI to generate reusable factory functions that accept override parameters for targeted edge-case customization.

Follow-Up Question: "What libraries in the JavaScript ecosystem complement AI-driven data generation?"

1. In risk-based test planning, which scenario should be categorized as P0 (Blocker / Critical)?

Key Concept: P0 scenarios represent critical revenue-generating paths without which the business cannot function. End-to-end checkout is a quintessential P0 scenario.

2. Why should automated test suites avoid using real customer production data?

Key Concept: Exposing real customer information violates privacy laws and introduces dangerous operational risks, such as accidental real-world communications or charges.

3. What is the purpose of the Test Data Factory pattern in test automation?

Key Concept: A Test Data Factory programmatically generates fresh, isolated, and valid data objects for every test, avoiding state conflicts between parallel test runs.

4. Which domain name is formally reserved by RFC standards specifically for safe testing and documentation?

Key Concept: RFC 2606 reserves example.com, example.net, and example.org for testing and documentation, guaranteeing no real emails will ever be routed to real people.

5. How do you tag a Playwright test with a requirement identifier in Playwright test configuration?

Key Concept: Playwright test options accept a tag array: test("title", { tag: ["@req-101"] }, async ({ page }) => { ... }).

6. What is the benefit of living requirements traceability in CI/CD pipelines?

Key Concept: Requirements traceability provides visibility into automation coverage, proving that critical business requirements are verified on every commit.

7. Which of the following is a classic temporal boundary edge case in date validation?

Key Concept: Same-day check-in and check-out is a classic boundary condition that many hotel systems reject, requiring explicit validation.

8. How can you run only tests matching the tag @smoke from the terminal in Playwright?

Key Concept: The --grep CLI option matches test titles and tags using string or regex filters.

9. Why should test data generators include dynamic elements like Date.now() or random suffixes?

Key Concept: Parallel test workers executing concurrently will collide if they attempt to create users with the same static username or email. Dynamic suffixes ensure uniqueness.

10. What is confirmation bias in the context of QA and test engineering?

Key Concept: Confirmation bias leads engineers to test happy paths that confirm their mental model, blind to adversarial paths and edge-case failures.